PYTHON PROGRAMMING – CASE STUDY REPORT

Name: Bhumula Jaswanth Narayana Reddy

Roll Number: 26000863

Branch: CSE (AIML)

Section: CSM-1

INTRODUCTION

Python is a high-level programming language that is easy to learn and use. It supports variables, data types, operators, conditional statements, loops, strings, lists, tuples, and functions. Python can be used to solve real-world problems through computational thinking and algorithm design.

This report presents ten Python case studies using basic programming concepts. Each case study includes a problem statement, objectives, assumptions, algorithm, flowchart description, pseudocode, Python source code, test cases, and sample output.

CASE STUDY 1: REAL-TIME WEATHER DATA AGGREGATOR

1. Problem Statement

To develop a Python program that stores weather information for different cities and calculates the average temperature, highest temperature, and lowest temperature.

2. Objectives

1. To store city names and temperatures.

2. To calculate average temperature.
3. To identify the highest and lowest temperatures.
4. To practice lists, loops, and conditional statements.

3. Assumptions

1. Temperatures are entered in Celsius.
2. At least one city is available.
3. The entered temperatures are valid numbers.

4. Algorithm

Step 1: Start the program.

Step 2: Enter the number of cities.

Step 3: Enter each city name and its temperature.

Step 4: Store the temperatures in a list.

Step 5: Calculate the average temperature.

Step 6: Find the maximum and minimum temperatures.

Step 7: Display the results.

Step 8: Stop the program.

5. Flowchart Description

(START)

|

v

/ Enter city names and temperatures /

|

v

[Store temperatures]

|

v

[Calculate average temperature]

|

v

[Find highest and lowest temperatures]

|

v

/ Display weather report /

|

v

(STOP)

6. Pseudocode

BEGIN

INPUT number of cities

FOR each city

INPUT city name and temperature

STORE temperature

END FOR

CALCULATE average temperature

FIND maximum temperature

FIND minimum temperature

DISPLAY results

END

7. Python Source Code

```
cities = ["Vijayawada", "Guntur", "Hyderabad"]
temperatures = [32, 35, 30]

average = sum(temperatures) / len(temperatures)

print("Weather Report")
print("Average Temperature:", round(average, 2), "C")
print("Highest Temperature:", max(temperatures), "C")
print("Lowest Temperature:", min(temperatures), "C")

for i in range(len(cities)):
    print(cities[i], ":", temperatures[i], "C")
```

8. Test Cases

Test Case 1: Temperatures are 32, 35, and 30.

Expected result: Average = 32.33°C, highest = 35°C, lowest = 30°C.

Test Case 2: Temperatures are 20, 25, and 15.

Expected result: Average = 20°C, highest = 25°C, lowest = 15°C.

9. Sample Output

Weather Report

Average Temperature: 32.333333333333336 C

Highest Temperature: 35 C

Lowest Temperature: 30 C

Vijayawada : 32 C

Guntur : 35 C

Hyderabad : 30 C

10. Conclusion

This case study demonstrates how Python lists, loops, and built-in functions can be used to analyze weather information.

CASE STUDY 2: E-COMMERCE INVENTORY AND BILLING SYSTEM

1. Problem Statement

To develop a Python program that stores product names, quantities, and prices and calculates the total bill.

2. Objectives

1. To maintain product information.
2. To calculate the cost of products.
3. To calculate the final bill.

4. To practice lists, arithmetic operators, and loops.

3. Assumptions

1. Product prices are positive.
2. Quantities are non-negative integers.
3. The bill is calculated without tax or discounts.

4. Algorithm

Step 1: Start the program.

Step 2: Store product names, prices, and quantities.

Step 3: Calculate the cost of each product.

Step 4: Add all product costs.

Step 5: Display the product details and total bill.

Step 6: Stop the program.

5. Flowchart Description

(START)

|

v

/ Enter product names, prices, quantities /

|

v

[Initialize total = 0]

|

v

[Calculate product cost]

|

v

[Add cost to total]

|

v

/ Display product bill /

|

v

/ Display total bill /

|

v

(STOP)

6. Pseudocode

BEGIN

STORE product names, prices, and quantities

SET total = 0

FOR each product

`CALCULATE price × quantity`

`ADD cost to total`

END FOR

DISPLAY total bill

END

7. Python Source Code

```
products = ["Keyboard", "Mouse", "Headphones"]
prices = [800, 400, 1200]
quantities = [2, 3, 1]

total = 0

print("E-Commerce Bill")

for i in range(len(products)):
    cost = prices[i] * quantities[i]
    total += cost
    print(products[i], "Cost:", cost)

print("Total Bill: Rs.", total)
```

8. Test Cases

Test Case 1: Prices are 800, 400, and 1200; quantities are 2, 3, and 1.

Expected result: Total bill = Rs. 4000.

Test Case 2: One product costs Rs. 500 and the quantity is 2.

Expected result: Total bill = Rs. 1000.

9. Sample Output

E-Commerce Bill

Keyboard Cost: 1600

Mouse Cost: 1200

Headphones Cost: 1200

Total Bill: Rs. 4000

10. Conclusion

This case study demonstrates how Python can calculate product costs and prepare a simple bill using lists and loops.

CASE STUDY 3: STUDENT GRADE ANALYSIS

1. Problem Statement

To develop a Python program that stores student marks, calculates the average, and determines the result.

2. Objectives

1. To store marks in a list.
2. To calculate total and average marks.
3. To determine pass or fail.
4. To practice conditional statements and arithmetic operators.

3. Assumptions

1. Marks are between 0 and 100.

2. A student must score at least 40 in every subject to pass.
3. The average is calculated using all subjects.

4. Algorithm

Step 1: Start the program.

Step 2: Enter the marks for each subject.

Step 3: Calculate the total marks.

Step 4: Calculate the average marks.

Step 5: Check whether the student passed all subjects.

Step 6: Display the total, average, and result.

Step 7: Stop the program.

5. Flowchart Description

(START)

|

v

/ Enter student marks /

|

v

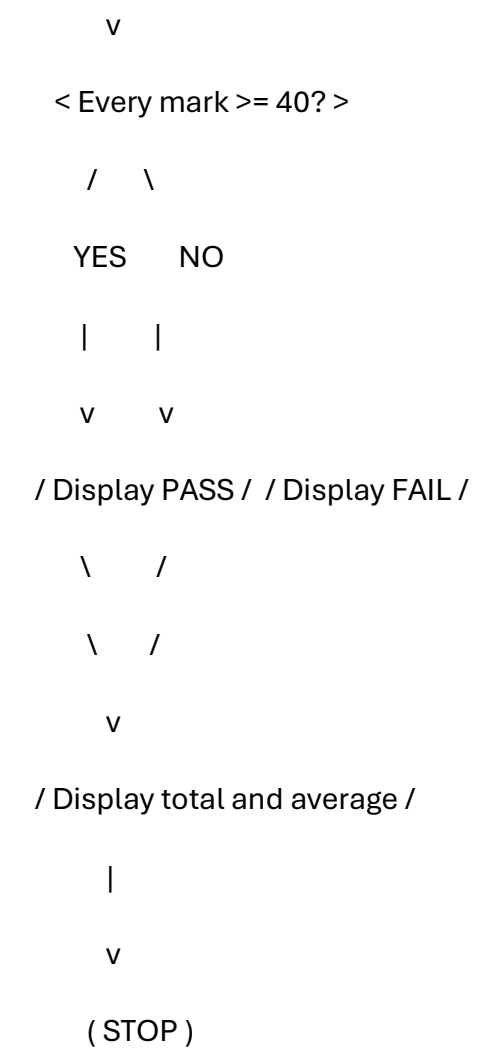
[Calculate total marks]

|

v

[Calculate average marks]

|



6. Pseudocode

BEGIN

INPUT marks

CALCULATE total

CALCULATE average

IF every mark is at least 40

DISPLAY PASS

ELSE

DISPLAY FAIL

END IF

DISPLAY total and average

END

7. Python Source Code

```
marks = [85, 78, 90, 66, 72]

total = sum(marks)
average = total / len(marks)

print("Total Marks:", total)
print("Average Marks:", average)

if all(mark >= 40 for mark in marks):
    print("Result: PASS")
else:
    print("Result: FAIL")
```

8. Test Cases

Test Case 1: Marks are 85, 78, 90, 66, and 72.

Expected result: Total = 391, average = 78.2, result = PASS.

Test Case 2: Marks are 80, 35, 70, 60, and 90.

Expected result: Result = FAIL.

9. Sample Output

Total Marks: 391

Average Marks: 78.2

Result: PASS

10. Conclusion

This case study demonstrates the use of lists, arithmetic operators, and conditional statements to analyze student performance.

CASE STUDY 4: FILE ORGANIZER

1. Problem Statement

To develop a Python program that classifies file names according to their extensions.

2. Objectives

1. To store file names.
2. To identify file extensions.
3. To classify files into categories.
4. To practice strings, lists, and conditional statements.

3. Assumptions

1. File names contain an extension.
2. The program classifies names only and does not move physical files.
3. Extensions are checked without considering uppercase or lowercase differences.

4. Algorithm

Step 1: Start the program.

Step 2: Store a list of file names.

Step 3: Read each file name.

Step 4: Identify its extension.

Step 5: Determine its category.

Step 6: Display the file name and category.

Step 7: Stop the program.

5. Flowchart Description

(START)

|

v

[Store file names]

|

v

/ Read file name /

|

v

[Identify extension]

|

v

< Extension type? >

/ | \



6. Pseudocode

```
BEGIN

STORE file names

FOR each file

FIND extension

IF extension is PDF

    CATEGORY = Document
```

```
ELSE IF extension is JPG or PNG
    CATEGORY = Image
ELSE IF extension is PY
    CATEGORY = Python File
ELSE
    CATEGORY = Other
END IF
DISPLAY file and category
END FOR
END
```

7. Python Source Code

```
files = ["report.pdf", "photo.jpg",
         "program.py", "notes.txt"]

for file in files:
    extension = file.split(".")[-1].lower()

    if extension == "pdf":
        category = "Document"
    elif extension in ["jpg", "png"]:
        category = "Image"
```

```
    elif extension == "py":
        category = "Python File"
    else:
        category = "Other"

    print(file, ":", category)
```

8. Test Cases

Test Case 1: File name is report.pdf.

Expected result: Document.

Test Case 2: File name is photo.jpg.

Expected result: Image.

Test Case 3: File name is program.py.

Expected result: Python File.

9. Sample Output

report.pdf : Document

photo.jpg : Image

program.py : Python File

notes.txt : Other

10. Conclusion

This case study demonstrates string operations and conditional statements for organizing file names by category.

CASE STUDY 5: QUIZ SYSTEM

1. Problem Statement

To develop a Python program that asks multiple-choice questions, checks answers, and calculates the final score.

2. Objectives

1. To display quiz questions.
2. To accept answers.
3. To check answers.
4. To calculate the score.

3. Assumptions

1. The quiz contains three questions.
2. Each correct answer receives one mark.
3. The program uses predefined answers.

4. Algorithm

Step 1: Start the program.

Step 2: Store the questions and correct answers.

Step 3: Display each question.

Step 4: Accept the user's answer.

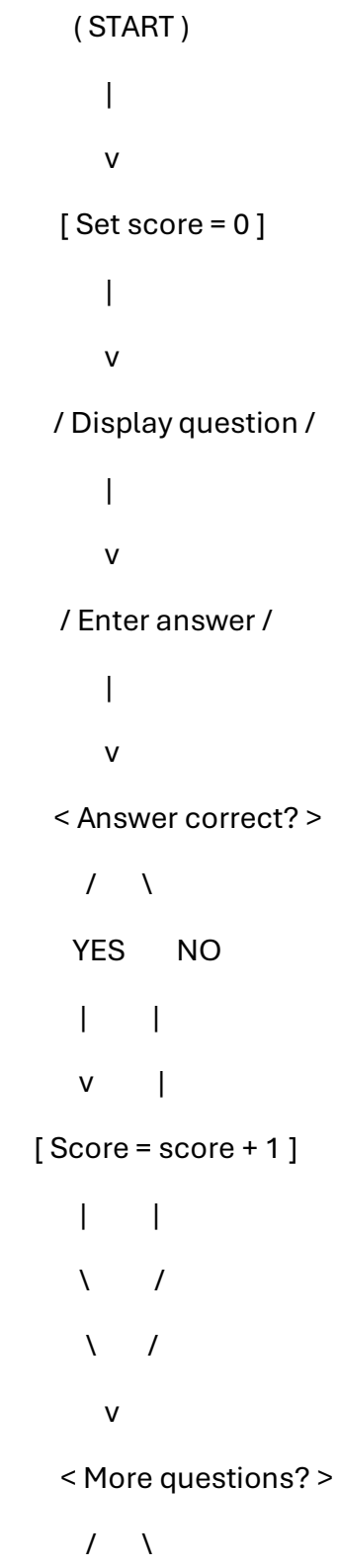
Step 5: Compare the answer with the correct answer.

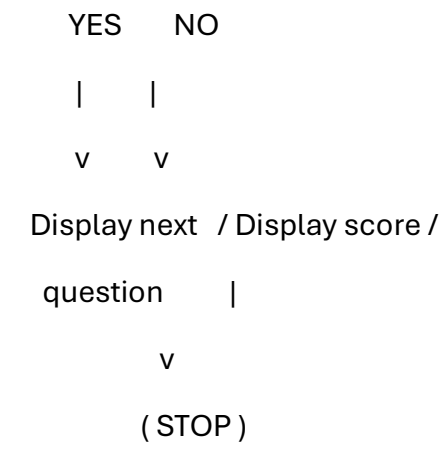
Step 6: Increase the score for each correct answer.

Step 7: Display the final score.

Step 8: Stop the program.

5. Flowchart Description





6. Pseudocode

```
BEGIN
SET score = 0
FOR each question
DISPLAY question
INPUT answer
IF answer is correct
    INCREASE score
END IF
END FOR
DISPLAY score
END
```

7. Python Source Code

```
questions = [
```

```
    ("What is the capital of India?", "Delhi"),
    ("Which language is used in this program?", "Python"),
    ("How many days are in a week?", "7")
]

score = 0

for question, correct_answer in questions:
    answer = input(question + " ")

    if answer.strip().lower() == correct_answer.lower():
        score += 1

print("Your Score:", score, "out of", len(questions))
```

8. Test Cases

Test Case 1: All answers are correct.

Expected result: Score = 3 out of 3.

Test Case 2: One answer is correct and two are incorrect.

Expected result: Score = 1 out of 3.

9. Sample Output

What is the capital of India? Delhi

Which language is used in this program? Python

How many days are in a week? 7

Your Score: 3 out of 3

10. Conclusion

This case study demonstrates the use of loops, strings, lists, and conditional statements to create a simple quiz system.

CASE STUDY 6: TASK TRACKER

1. Problem Statement

To develop a Python program that stores tasks and displays their completion status.

2. Objectives

1. To maintain a list of tasks.
2. To store task status.
3. To display pending and completed tasks.
4. To practice lists and loops.

3. Assumptions

1. Tasks are entered in advance.
2. Each task has a completion status.
3. The program displays the stored information.

4. Algorithm

Step 1: Start the program.

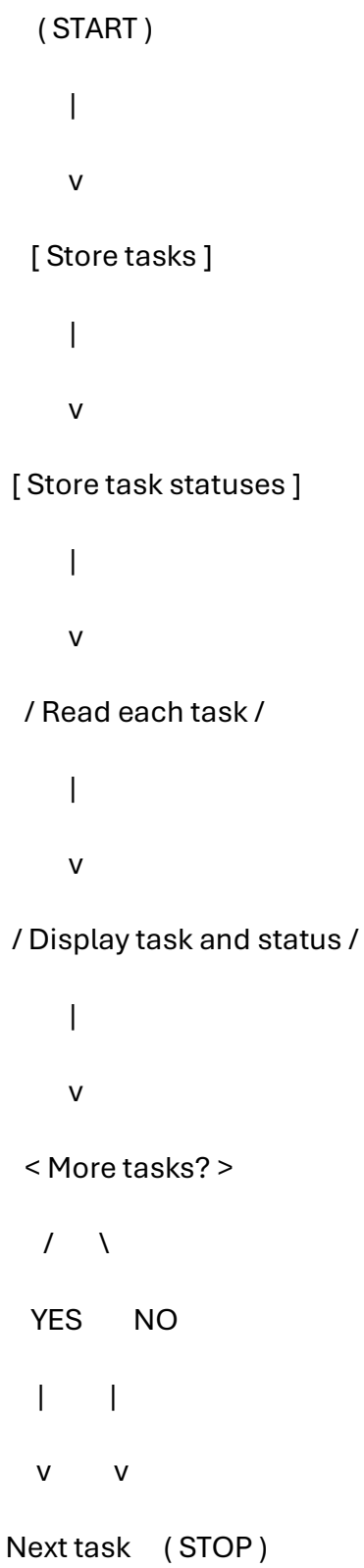
Step 2: Store task names.

Step 3: Store the status of each task.

Step 4: Display each task and its status.

Step 5: Stop the program.

5. Flowchart Description



6. Pseudocode

BEGIN

STORE tasks and statuses

FOR each task

DISPLAY task name and status

END FOR

END

7. Python Source Code

```
tasks = [
    "Complete assignment",
    "Study Python",
    "Prepare report"
]

statuses = ["Completed", "Pending", "Pending"]

print("Task Tracker")

for i in range(len(tasks)):
    print(tasks[i], "-", statuses[i])
```

8. Test Cases

Test Case 1: One task is completed and two are pending.

Expected result: One completed task and two pending tasks are displayed.

Test Case 2: All tasks are completed.

Expected result: Every task displays Completed.

9. Sample Output

Test Case 2: Expense is 10 USD using the sample rate of 83 INR per USD.

Expected result: Rs. 830.

9. Sample Output

100 INR = 100 INR

10 USD = 830 INR

5 EUR = 450 INR

Total Expense in INR: 1380

10. Conclusion

This case study demonstrates currency conversion and expense calculation using Python dictionaries, tuples, lists, and loops.

OVERALL CONCLUSION

The ten case studies demonstrate how Python programming concepts can be applied to practical problems. Lists and tuples store data, strings process text, operators perform calculations, loops repeat operations, and conditional statements make decisions.

These case studies improve computational thinking, algorithm development, problem-solving ability, and basic Python programming skills.

REFERENCES

1. Python official documentation: https://docs.python.org/3/
2. Python programming class notes.
3. Introductory programming and computational thinking materials.

10. Conclusion

This case study demonstrates how lists and loops can be used to manage and display tasks.

CASE STUDY 7: LOG FILE PARSER

1. Problem Statement

To develop a Python program that analyzes log messages and counts the number of informational, warning, and error messages.

2. Objectives

1. To store log messages.
2. To identify message types.
3. To count each type.
4. To practice strings, lists, and loops.

3. Assumptions

1. Every log message contains a known type.
2. The types used are INFO, WARNING, and ERROR.
3. The messages are already stored in a list.

4. Algorithm

Step 1: Start the program.

Step 2: Store log messages.

Step 3: Initialize counters for INFO, WARNING, and ERROR.

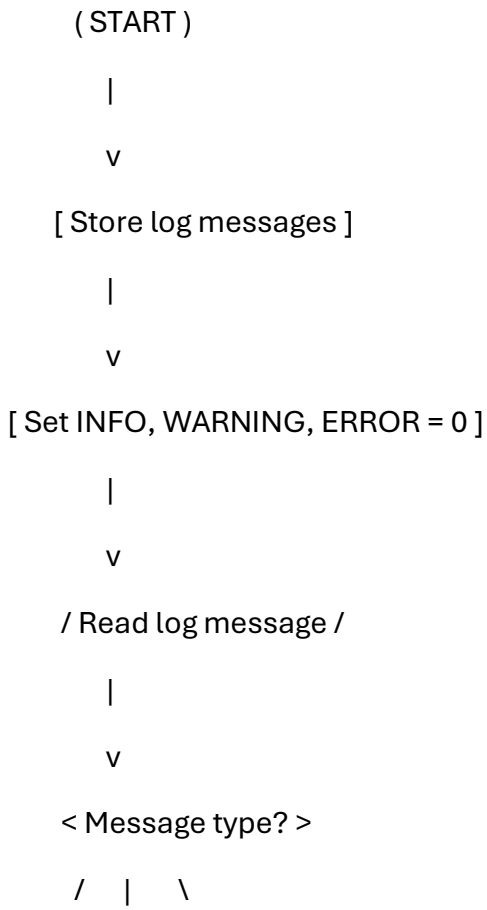
Step 4: Examine each message.

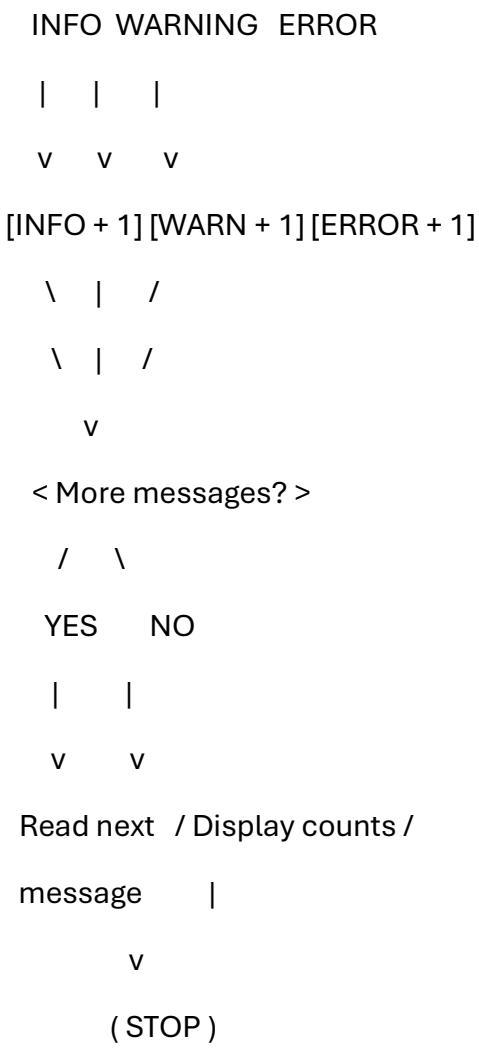
Step 5: Increase the matching counter.

Step 6: Display the totals.

Step 7: Stop the program.

5. Flowchart Description





6. Pseudocode

```
BEGIN

STORE log messages

SET INFO = 0

SET WARNING = 0

SET ERROR = 0

FOR each message
```

```
IF message contains INFO
    INCREASE INFO
ELSE IF message contains WARNING
    INCREASE WARNING
ELSE IF message contains ERROR
    INCREASE ERROR
END IF
END FOR
DISPLAY all counters
END
```

7. Python Source Code

```
logs = [
    "INFO: Program started",
    "WARNING: Low memory",
    "ERROR: File not found",
    "INFO: Program completed"
]

info = 0
warning = 0
error = 0
```

```
for log in logs:
    if "INFO" in log:
        info += 1
    elif "WARNING" in log:
        warning += 1
    elif "ERROR" in log:
        error += 1

print("INFO messages:", info)
print("WARNING messages:", warning)
print("ERROR messages:", error)
```

8. Test Cases

Test Case 1: Two INFO messages, one WARNING, and one ERROR.

Expected result: INFO = 2, WARNING = 1, ERROR = 1.

Test Case 2: Three ERROR messages.

Expected result: ERROR = 3.

9. Sample Output

INFO messages: 2

WARNING messages: 1

ERROR messages: 1

10. Conclusion

This case study demonstrates how Python string operations, loops, and counters can be used to analyze log messages.

CASE STUDY 8: PORTFOLIO RISK ANALYSIS

1. Problem Statement

To develop a Python program that stores investment values and calculates the total investment, average value, and highest investment.

2. Objectives

1. To store investment values.
2. To calculate the total investment.
3. To calculate the average value.
4. To identify the highest investment.

3. Assumptions

1. All investment values are non-negative.
2. The program uses sample data.
3. The analysis does not predict actual investment risk or future returns.

4. Algorithm

Step 1: Start the program.

Step 2: Store investment values.

Step 3: Calculate the total.

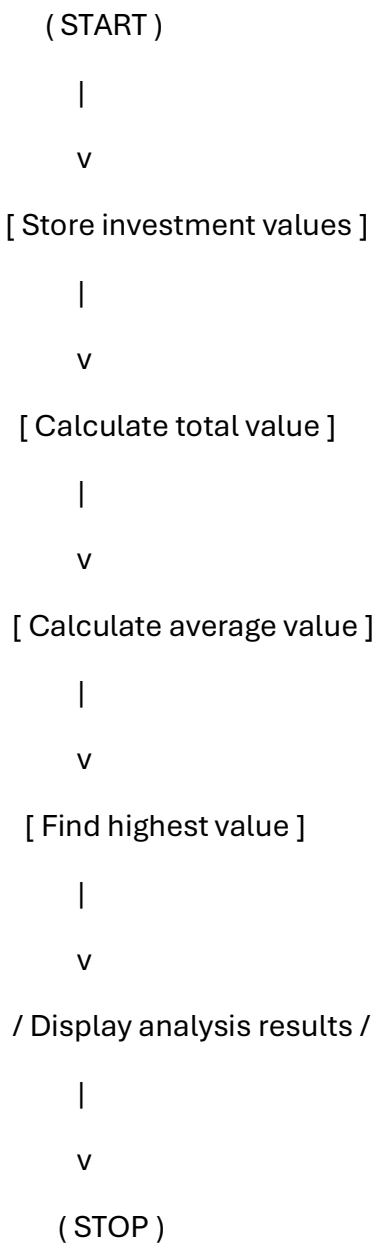
Step 4: Calculate the average.

Step 5: Find the highest value.

Step 6: Display the results.

Step 7: Stop the program.

5. Flowchart Description



6. Pseudocode

BEGIN

STORE investment values

CALCULATE total

CALCULATE average

FIND maximum value

DISPLAY total, average, and maximum

END

7. Python Source Code

```
investments = [5000, 8000, 3000, 10000]

total = sum(investments)
average = total / len(investments)
highest = max(investments)

print("Total Investment:", total)
print("Average Investment:", average)
print("Highest Investment:", highest)
```

8. Test Cases

Test Case 1: Investments are 5000, 8000, 3000, and 10000.

Expected result: Total = 26000, average = 6500, highest = 10000.

Test Case 2: Investments are 1000, 2000, and 3000.

Expected result: Total = 6000, average = 2000, highest = 3000.

9. Sample Output

Total Investment: 26000

Average Investment: 6500.0

Highest Investment: 10000

10. Conclusion

This case study demonstrates basic numerical analysis using Python lists and built-in functions.

CASE STUDY 9: LIBRARY MANAGEMENT SYSTEM

1. Problem Statement

To develop a Python program that stores book information and checks whether a requested book is available in the library.

2. Objectives

1. To store book names.
2. To search for a book.
3. To display whether the book is available.
4. To practice lists and conditional statements.

3. Assumptions

1. Book names are stored in a list.
2. The search ignores uppercase and lowercase differences.
3. The program checks availability but does not issue or return books.

4. Algorithm

Step 1: Start the program.

Step 2: Store the available book names.

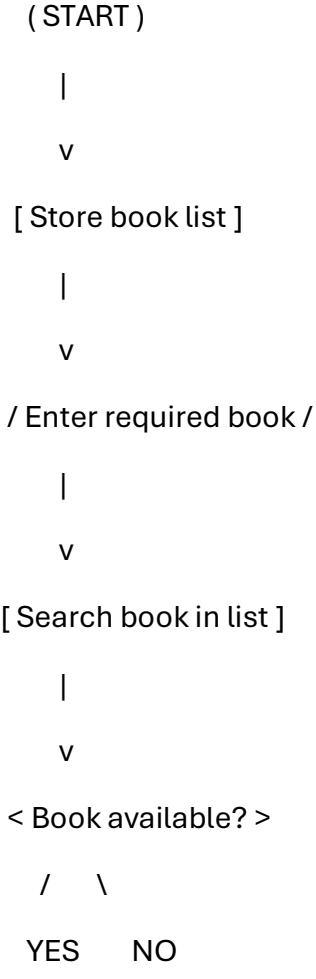
Step 3: Enter the name of the required book.

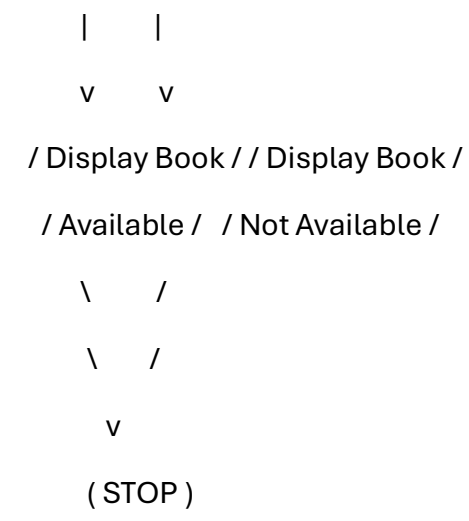
Step 4: Compare the requested book with the stored list.

Step 5: Display whether the book is available.

Step 6: Stop the program.

5. Flowchart Description





6. Pseudocode

```
BEGIN

STORE available books

INPUT required book

IF required book is in the list

DISPLAY Book Available

ELSE

DISPLAY Book Not Available

END IF

END
```

7. Python Source Code

```
books = [
    "Python Basics",
    "Data Science",
    "Web Development",
    "Computer Networks"
]

required_book = input("Enter book name: ").strip().lower()

available_books = [book.lower() for book in books]

if required_book in available_books:
    print("Book Available")
else:
```

ilab print("Book Not Ava le")

8. Test Cases

Test Case 1: Requested book is Python Basics.

Expected result: Book Available.

Test Case 2: Requested book is Java Programming.

Expected result: Book Not Available.

9. Sample Output

Enter book name: Python Basics

Book Available

10. Conclusion

This case study demonstrates how Python lists and conditional statements can be used for a simple library book search.

CASE STUDY 10: MULTI-CURRENCY EXPENSE CALCULATOR

1. Problem Statement

To develop a Python program that converts expenses from different currencies into Indian rupees and calculates the total expense.

2. Objectives

1. To store expenses in different currencies.
2. To use exchange rates for conversion.

3. To calculate the total expense in Indian rupees.
4. To practice dictionaries, lists, loops, and arithmetic operators.

3. Assumptions

1. The exchange rates are sample values for demonstration.
2. The rates do not represent live exchange rates.
3. Every expense uses a supported currency.

4. Algorithm

Step 1: Start the program.

Step 2: Store sample exchange rates.

Step 3: Store the expenses and their currencies.

Step 4: Convert each expense into Indian rupees.

Step 5: Add the converted amounts.

Step 6: Display the total expense.

Step 7: Stop the program.

5. Flowchart Description

(START)

|

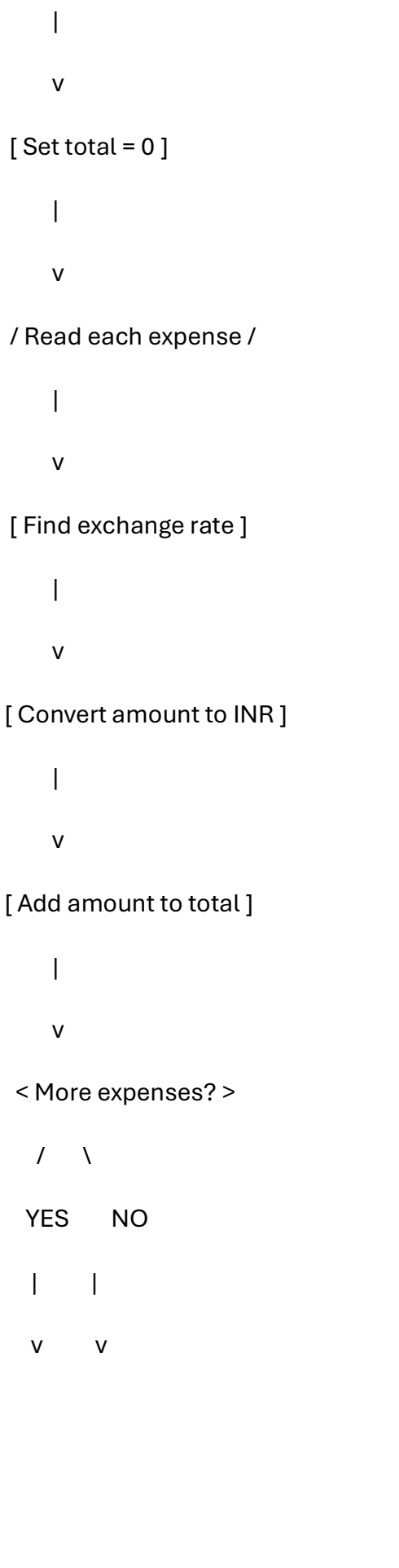
v

[Store exchange rates]

|

v

[Store expenses]


[Set total = 0]
/ Read each expense /
[Find exchange rate]
[Convert amount to INR]
[Add amount to total]
< More expenses? >
YES
NO

Next expense / Display total /

|

v

(STOP)

6. Pseudocode

BEGIN

STORE exchange rates

STORE expenses

SET total = 0

FOR each expense

FIND exchange rate

CONVERT expense to INR

ADD converted amount to total

END FOR

DISPLAY total expense

END

7. Python Source Code

```
rates = {
    "INR": 1,
    "USD": 83,
    "EUR": 90
```

```
}

expenses = [
    (100, "INR"),
    (10, "USD"),
    (5, "EUR")
]

total = 0

for amount, currency in expenses:
    converted = amount * rates[currency]
    total += converted
    print(amount, currency, "=", converted, "INR")

print("Total Expense in INR:", total)
```

8. Test Cases

Test Case 1: Expenses are 100 INR, 10 USD, and 5 EUR using the sample rates.

Expected result: Total = Rs. 100 + Rs. 830 + Rs. 450 = Rs. 1380.

Test Case 2: Expense is 10 USD using the sample rate of 83 INR per USD.

Expected result: Rs. 830.

9. Sample Output

100 INR = 100 INR

10 USD = 830 INR

5 EUR = 450 INR

Total Expense in INR: 1380

10. Conclusion

This case study demonstrates currency conversion and expense calculation using Python dictionaries, tuples, lists, and loops.

OVERALL CONCLUSION

The ten case studies demonstrate how Python programming concepts can be applied to practical problems. Lists and tuples store data, strings process text, operators perform calculations, loops repeat operations, and conditional statements make decisions.

These case studies improve computational thinking, algorithm development, problem-solving ability, and basic Python programming skills.